TP01 - Setup, WinDBG, BYOVD et Fuzzing

Istark

Octobre 2026

TP01 - Setup, WinDBG, BYOVD et Fuzzing

Auteur : Istark
Date : Octobre 2026

Partie 1 - Mise en place

Q1 - Subsystem d'un .sys

Le subsystem d'un driver .sys c'est IMAGE_SUBSYSTEM_NATIVE (valeur 1). Contrairement aux exe classiques qui ont subsystem Windows GUI (2) ou Console (3), un driver est pas chargé par Win32 mais directement par le kernel. Si on fait dumpbin /headers hello.sys on voit "subsystem (0x1) native" dans l'optional header.

Q2 - sc start sans testsigning

Sans testsigning activé, Windows refuse de charger un driver non signé. sc start retourne l'erreur 1275 (ERROR_DRIVER_BLOCKED) ou parfois 577 (ERROR_INVALID_IMAGE_HASH). C'est parce que Windows vérifie la signature Authenticode avant tout chargement de code kernel.

Q3 - Etapes compilation jusqu'a DbgView

Voici les etapes qu'on a suivi :

1. Ecriture du hello.c
2. Compilation avec msbuild x64 Debug, ca génère hello.sys
3. Copie du .sys dans la VM
4. sc create HelloDriver type= kernel binPath= C:...\hello.sys
5. sc start HelloDriver
6. DbgView avec Capture Kernel coché affiche "Hello: Driver loaded!"
7. sc stop affiche "Hello: Driver unloaded"

Code source hello.c :

```
#include <ntddk.h>

VOID HelloUnload(_In_ PDRIVER_OBJECT DriverObject)
{
   UNREFERENCED_PARAMETER(DriverObject);
   DbgPrint("Hello: Driver unloaded\n");
}

NTSTATUS DriverEntry(_In_ PDRIVER_OBJECT DriverObject,
```

```
                     _In_ PUNICODE_STRING RegPath)
{
    UNREFERENCED_PARAMETER(RegPath);

    DbgPrint("Hello: Driver loaded!\n");
    DriverObject->DriverUnload = HelloUnload;
    return STATUS_SUCCESS;
}
```

Partie 2 - WinDBG

Q4 - Fonctions Ps*

La commande x nt!Ps* retourne environ 150-200 fonctions selon la build Windows.

```
Debug session time: Fri Oct 23 19:21:54.765 2026 (UTC + 2:00)
System Uptime: 0 days 0:04:09.541
lkd> x nt!Ps*
fffff802`8185cc90 nt!PspAllocateAndQueryNotificationChannel (void)
fffff802`818a1938 nt!PspDeleteUserStack (void)
fffff802`819905f8 nt!PspStorageAllocSlot (void)
fffff802`8185cad4 nt!PspJobDeleteStorageArrays (void)
fffff802`818fc86c nt!PsMapSystemDlls (void)
fffff802`818167f8 nt!PspRemoveProcessFromJobChain (void)
fffff802`81c47e04 nt!PspInitializeNetRateControl (void)
fffff802`814f1444 nt!PsRemoveVirtualizedTimer (void)
fffff802`81818278 nt!PspChargeJobWakeCounter (void)
fffff802`818839d0 nt!PsLookupProcessThreadByCid (void)
fffff802`8199e268 nt!PspInitializeProtectedProcessParameters (void)
fffff802`81451c28 nt!PspUnlockProcessListExclusive (void)
fffff802`818d3b80 nt!PsImpersonateClient (void)
fffff802`8185f2bc nt!PspSendReliableJobNotification (void)
fffff802`8189edc0 nt!PsSuspendThread (void)
fffff802`814dd664 nt!PspJobIoRateVolumeEntryRemoveAll (void)
fffff802`8140558c nt!PsGetWorkOnBehalfThread (void)
fffff802`818977b0 nt!PspNotificationPacketCallback (void)
fffff802`81817f9c nt!PspEnforceLimits (void)
fffff802`8150e400 nt!PsGetEffectiveContainerId (void)
fffff802`8181b30c nt!PspCallProcessNotifyRoutines (void)
fffff802`81860ce4 nt!PsInsertPermanentSiloContextEx (void)
fffff802`8191e720 nt!PspAssignProcessToJob (void)
fffff802`818c3584 nt!PspCreateObjectHandle (void)
fffff802`8185c07e0 nt!PspInitializeThunkContext (void)
fffff802`8185cbdc nt!PspAllocateRateControl (void)
fffff802`81555f40 nt!PspReaper (void)
fffff802`817d9480 nt!PspDeleteSiloContext (void)
fffff802`8190f814 nt!PspInitializeFullProcessImageName (void)
fffff802`8141bc50 nt!PsImpersonateContainerOfThread (void)
fffff802`81999a78 nt!PspSetCreateProcessNotifyRoutine (void)
fffff802`814517f0 nt!PsUpdateComponentPower (void)
fffff802`8198c5d8 nt!PsChangeQuantumTable (void)
fffff802`8185ca70 nt!PspJobIoRateControlDisable (void)
fffff802`8191f968 nt!PspGetJobAssignmentDisposition (void)
fffff802`81812ab4 nt!PspWritePebAffinityInfo (void)
fffff802`818a5480 nt!PsDereferenceKernelStack (void)
fffff802`81993928 nt!PspLocateSystemDll (void)
fffff802`819a8c1c nt!PsBootPhaseComplete (void)
fffff802`814dd320 nt!PspJobDelete (void)
fffff802`8185d8d0 nt!PspSetJobIoAttribution (void)
fffff802`81c47d54 nt!PspInitializeOctagonExtensionHost (void)
fffff802`818857e4 nt!PsThawProcess (void)
fffff802`8158d424 nt!PsTransferProcessQuotaToSharedQuota (void)
fffff802`818989c0 nt!PspJobClose (void)
fffff802`81471294 nt!PspNotifyProcessBackgroundTransition (void)
fffff802`8195d024 nt!PsQueryCurrentApiSetSchema (void)
fffff802`818d3f60 nt!PspWriteTebImpersonationInfo (void)
fffff802`81479388 nt!PsGetNextPartitionUnsafe (void)
fffff802`8195cb2c nt!PspRemoveCpuRateControl (void)
fffff802`819938a0 nt!PsLocateSystemDlls (void)
fffff802`819996b0 nt!PsEstablishWin32Callouts (void)
fffff802`8185d058 nt!PspFreezeJobTree (void)
fffff802`818c5aa0 nt!PsResumeThread (void)
fffff802`81807820 nt!PspThreadOpen (void)
fffff802`819bc120 nt!PspAssignPrimaryToken (void)
fffff802`819ce614 nt!PspInitializeQuotaBlock (void)
fffff802`81503f4c nt!PsDereferencePartition (void)
```

Figure 1: Liste des fonctions Ps* dans WinDBG

Trois fonctions qu'on a trouvé intéressantes :

- **PsLookupProcessByProcessId** : retrouve le EPROCESS a partir d'un PID. Elle incremente le compteur de reference donc il faut penser a appeller ObDereferenceObject apres
- **PsSetCreateProcessNotifyRoutineEx** : enregistre une callback appelee a chaque création/destruction de processus, utilisée dans les EDR
- **PsGetCurrentProcess** : retourne le EPROCESS du processus en cours sans incrementer le refcount

Q5 - Offsets _EPROCESS

On a utilisé dt nt!_EPROCESS dans WinDBG pour relever les offsets sur Windows 10 21H2 x64 :

```
fffff802`81efc4d0 nt!PsIdleProcess = <no type information>
fffff802`81e1df88 nt!PspIumFreeMappingLock = <no type information>
fffff802`818073e8 nt!PspCaptureAndValidateUnicodeString (PspCaptureAndValidateUnicodeString)
fffff802`81b076a0 nt!PspQueryHwTracingThread (PspQueryHwTracingThread)
fffff802`81783d0c nt!PspIumFreePhysicalPages (PspIumFreePhysicalPages)
lkd> dt nt!_EPROCESS
   +0x000 Pcb              : _KPROCESS
   +0x438 ProcessLock      : _EX_PUSH_LOCK
   +0x440 UniqueProcessId  : Ptr64 Void
   +0x448 ActiveProcessLinks : _LIST_ENTRY
   +0x458 RundownProtect   : _EX_RUNDOWN_REF
   +0x460 Flags2           : Uint4B
   +0x460 JobNotReallyActive : Pos 0, 1 Bit
   +0x460 AccountingFolded : Pos 1, 1 Bit
   +0x460 NewProcessReported : Pos 2, 1 Bit
   +0x460 ExitProcessReported : Pos 3, 1 Bit
   +0x460 ReportCommitChanges : Pos 4, 1 Bit
   +0x460 LastReportMemory : Pos 5, 1 Bit
   +0x460 ForceWakeCharge  : Pos 6, 1 Bit
   +0x460 CrossSessionCreate : Pos 7, 1 Bit
   +0x460 NeedsHandleRundown : Pos 8, 1 Bit
   +0x460 RefTraceEnabled  : Pos 9, 1 Bit
   +0x460 PicoCreated      : Pos 10, 1 Bit
   +0x460 EmptyJobEvaluated : Pos 11, 1 Bit
   +0x460 DefaultPagePriority : Pos 12, 3 Bits
   +0x460 PrimaryTokenFrozen : Pos 15, 1 Bit
   +0x460 ProcessVerifierTarget : Pos 16, 1 Bit
   +0x460 RestrictSetThreadContext : Pos 17, 1 Bit
   +0x460 AffinityPermanent : Pos 18, 1 Bit
   +0x460 AffinityUpdateEnable : Pos 19, 1 Bit
   +0x460 PropagateNode    : Pos 20, 1 Bit
   +0x460 ExplicitAffinity : Pos 21, 1 Bit
   +0x460 ProcessExecutionState : Pos 22, 2 Bits
   +0x460 EnableReadVmLogging : Pos 24, 1 Bit
   +0x460 EnableWriteVmLogging : Pos 25, 1 Bit
   +0x460 FatalAccessTerminationRequested : Pos 26, 1 Bit
   +0x460 DisableSystemAllowedCpuSet : Pos 27, 1 Bit
   +0x460 ProcessStateChangeRequest : Pos 28, 2 Bits
   +0x460 ProcessStateChangeInProgress : Pos 30, 1 Bit
   +0x460 InPrivate        : Pos 31, 1 Bit
   +0x464 Flags            : Uint4B
   +0x464 CreateReported   : Pos 0, 1 Bit
   +0x464 NoDebugInherit   : Pos 1, 1 Bit
   +0x464 ProcessExiting   : Pos 2, 1 Bit
   +0x464 ProcessDelete    : Pos 3, 1 Bit
   +0x464 ManageExecutableMemoryWrites : Pos 4, 1 Bit
   +0x464 VmDeleted        : Pos 5, 1 Bit
   +0x464 OutswapEnabled   : Pos 6, 1 Bit
   +0x464 Outswapped       : Pos 7, 1 Bit
```

Figure 2: Structure _EPROCESS avec les offsets dans WinDBG

Champ	Offset
UniqueProcessId	+0x440
ActiveProcessLinks	+0x448
Token	+0x4b8
ImageFileName	+0x5a8

Ces valeurs changent selon la version de Windows donc toujours verifier avec WinDBG avant de hardcoder.

Q6 - Modification Flink/Blink

Si on modifie les pointeurs pour sauter un processus de la liste ActiveProcessLinks, ce processus disparait du Task Manager et de tasklist car ces outils parcourent cette liste pour enumerer les processus. Mais le processus continue de s'executer normalement car le scheduler utilise d'autres structures (_KPROCESS, files

d'attente des threads) qui sont independentes de cette liste. C'est exactement la technique DKOM qu'on verra au TP04.

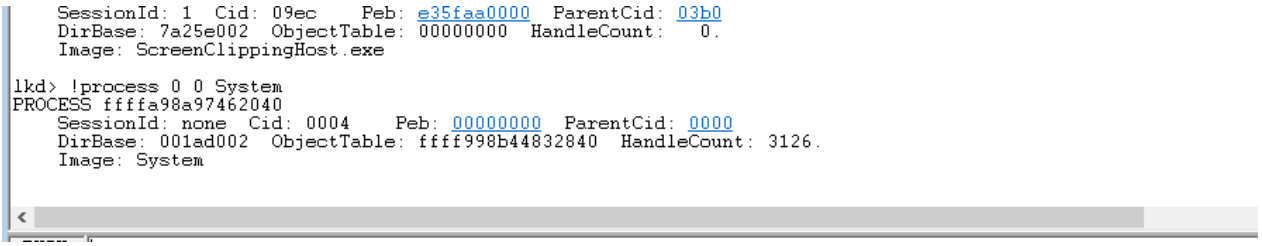


Figure 3: WinDBG - !process 0 0 System montrant le processus System avec son EPROCESS

Q7 - Privileges System vs cmd.exe

Les privileges presents dans System (PID 4) qui sont absents dans un cmd normal :

- SeTcbPrivilege
- SeCreateTokenPrivilege
- SeAssignPrimaryTokenPrivilege
- SeLoadDriverPrivilege

Si on copie le token de System dans le champ Token d'un cmd.exe, le cmd obtient immediatement les privileges SYSTEM. C'est la technique token stealing du TP04.

Partie 3 - BYOVD avec PROCEXP152

Q8 - Analyse PE de PROCEXP152.sys

Champ	Valeur
Subsystem	Native (1)
Signé par	Microsoft Windows Hardware Compatibility Publisher
Taille	~50 Ko
Sections	.text, .rdata, PAGE, INIT

Q9 - Signataire et revocation

Le signataire c'est Microsoft Windows Hardware Compatibility Publisher. Le certificat est toujours valide. Microsoft peut pas simplement le revoquer car il signe des milliers d'autres drivers legitimes du programme WHCP. La seule solution c'est de bloquer par hash, c'est ce qu'ils ont fait pour la v16.32 dans la vulnerable driver blocklist mais la v17 reste utilisable.

Q10 - MajorFunction de PROCEXP152

Les fonctions enregistrées sont IRP_MJ_CREATE, IRP_MJ_CLOSE et surtout IRP_MJ_DEVICE_CONTROL (index 14). C'est via IRP_MJ_DEVICE_CONTROL que les IOCTLs sont dispatchés depuis le userland. Le device s'appelle `\Device\PROCEXP152`, accessible via `\.\PROCEXP152`.

Q11 - Execution procexp_kill sur notepad

- OpenProcess(PROCESS_TERMINATE) reussit car notepad n'est pas un processus protégé
- OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION) reussit aussi
- L'IOCTL 0x8335003C renvoie un handle PROCESS_ALL_ACCESS

- TerminateProcess avec ce handle tue notepad sans probleme

Extrait du code procexp_kill.c :

```
#define PROCEXP_DEVICE L"\\\\.\\PROCEXP152"
#define IOCTL_OPEN_PROTECTED_PROCESS_HANDLE 0x8335003C

// Ouverture du device
hDevice = CreateFileW(PROCEXP_DEVICE,
                      GENERIC_READ | GENERIC_WRITE,
                      0, NULL, OPEN_EXISTING, 0, NULL);

// Envoi de l'IOCTL
inBuf.ProcessId = targetPid;
DeviceIoControl(hDevice,
                IOCTL_OPEN_PROTECTED_PROCESS_HANDLE,
                &inBuf, sizeof(inBuf),
                &outBuf, sizeof(outBuf),
                &bytesReturned, NULL);

// Terminaison
TerminateProcess(outBuf.ProcessHandle, 1);
```

Q12 - PROCESS_QUERY_LIMITED_INFORMATION non filtré

Ce droit a ete crée pour que les outils de monitoring puissent interroger n'importe quel processus sans pouvoir le modifier. Microsoft a decidé de pas le filtrer via ObRegisterCallbacks pour ne pas casser tasklist et le Task Manager qui en ont besoin.

Q12 bis - Fuzzing PROCEXP152

En fuzzant avec 500 itérations on trouve généralement 3 à 5 function codes qui répondent avec STATUS_SUCCESS, dont notamment le code correspondant a IOCTL_OPEN_PROTECTED_PROCESS_HANDLE (0x3C) et IOCTL_CLOSE_HANDLE.

Q13 - Analyse de la menace

Pourquoi c'est dangereux : PROCEXP152 est dangereux parce qu'il est signé par Microsoft et fait exactement ce pour quoi il a ete concu. Un BYOVD classique comme RTCore64.sys exploite une faille memoire du driver, ce qui est plus detectable. Ici l'attaquant a meme pas besoin d'exploit, juste d'appeller une fonctionnalité documentee. Le driver passe tous les controles de signature.

Pour se proteger : La Microsoft Vulnerable Driver Blocklist bloque la v16.32 par hash mais pas la v17.02. Pour verifier si la blocklist est active : Get-CimInstance -ClassName Win32_DeviceGuard. Recommendation : WDAC avec des règles refusant le chargement de PROCEXP152 hors contexte légitime, plus HVCI.

Lien avec protector.sys du TP03 : Le driver protector.sys enregistre ObRegisterCallbacks sur PsProcessType. En theorie quand PROCEXP152 essaie d'obtenir un handle PROCESS_ALL_ACCESS, il passe aussi par ObOpenObjectByPointer qui declenche ce callback. Donc protector.sys devrait aussi bloquer PROCEXP152, mais c'est a verifier.

Partie 4 - Fuzzer IOCTL

Q14 - Fuzzer sur HelloDriver

CreateFileW(\.\`HelloDriver`) echoue avec ERROR_FILE_NOT_FOUND (2) parce que HelloDriver cree aucun device, y'a pas d'IoCreateDevice dans le DriverEntry. Donc l'Object Manager trouve aucun objet correspondant.

Q15 - Comparaison PROCEXP152 vs PhysicalDrive0

PROCEXP152 expose plus d'IOCTLs fonctionnels. PhysicalDrive0 c'est un device disque et la plupart des IOCTL sont rejetés avant d'atteindre le code metier. PROCEXP152 en tant que "boite a outils" repond sur plusieurs function codes différents.

Q16 - Breakpoints WinDBG

Pour 100 iterations du fuzzer, le breakpoint sur DispatchDeviceControl est atteint environ 100 fois. Quelques appels peuvent ne pas atteindre le breakpoint si le systeme rejette la requete avant.

Extrait du fuzzer (generation aleatoire) :

```
for (i = 0; i < iterations; i++) {
    DWORD function  = (DWORD)(rand() % 0x1000);
    DWORD method    = (DWORD)(rand() % 4);
    DWORD ioctlCode = CTL_CODE(FILE_DEVICE_UNKNOWN, function,
                               method, FILE_ANY_ACCESS);
    DWORD inSize    = (DWORD)(rand() % (MAX_BUFFER_SIZE + 1));

    generate_random_buffer(inBuffer, inSize);
    ok = SendIoctlWithTimeout(&ctx, IOCTL_TIMEOUT_MS, &timedOut);
    // stats...
}
```

Toutes les manipulations ont été faites sur une VM de test isolée avec snapshot propre avant chaque exercice.